BaiZe: Cost-Efficient Scripting and Waveform Perception for EDA Workflow Orchestration

Abstract*—Agentic LLMs for electronic design automation (EDA) are already deployed in-house, with models such as DeepSeek-V4-Flash and GLM-5.3-Flash serving as the coordinating master. Cost, not feasibility, is now the binding constraint: every EDA subtask routed to a frontier master consumes tokens and serving budget that scale with design-flow volume. We present BaiZe, a ≈2.2B hybrid state-space agentic model built from scratch as a cost-efficient EDA subagent beneath such a master: it receives subtasks in natural language and drives EDA tools in their native scripting languages. Its 56 layers interleave Mamba-2 state-space and MLP layers with four attention layers, so only the latter carry a session-growing state; decoding is linear-time and per-subtask serving cost is governed by session memory rather than prompt throughput. A matched from-scratch comparison against a dense transformer selects this architecture (11.6% fewer parameters, lower loss at a matched 1k-step budget, and $+10{:}5\%$ prefill throughput), and a budget-constrained search fixes the pre-training recipe, now scaled to a 2.94T-token production run. The backbone now stands at training loss 2.151 after 30,660 iterations (≈598K tok/s), and the encoder reaches 19.76% / 20.27% ImageNet linear probe at 55.3M / 176M samples; a larger ViT-L/14 tower reaches $68{:}2\%$ on an internal protocol, yet the MLLM stage averages $26{:}1\%$ over 11 multimodal benchmarks, showing that encoder gains do not transfer downstream at the 224-px single-tile setting. On a 449-task EDA-scripting benchmark spanning three languages and four commercial runtimes [?], we test whether BaiZe can offload subtasks from the deployed Flash masters at comparable task accuracy and materially lower serving cost, and whether a from-scratch vision encoder's 196-token-per-image budget suffices for detail-dense layout, schematic, and waveform views. Later stages are specified as protocols with pre-specified criteria; their measurements are in progress, and we release the full recipe and evaluation protocol.

***Index Terms*—small language model, agentic LLM, hybrid state-space architecture, EDA subagent, cost-efficient inference, multimodal grounding**

I. Introduction

Large language models have recently demonstrated agentic capabilities—planning, tool invocation, and long-horizon reasoning—that make them attractive cores for intelligent assistants in electronic design automation (EDA). Such assistants are no longer hypothetical: agentic EDA flows are already deployed in-house, with Flash-tier masters such as DeepSeek-V4-Flash and GLM-5.3-Flash coordinating the flow. The binding constraint is therefore cost, dominated by two terms: every subtask routed to a master consumes tokens and serving budget that scale with flow volume, and because an agentic session accumulates context, serving cost is driven by the memory that grows with the session rather than by prompt throughput. A small, locally served subagent caps the cost of every subtask it takes over by pricing it at a local 2.2B model rather than at a Flash master. Building such a model *from scratch* as a production subagent, however, remains an under-documented engineering problem.

a) Contributions.: We present BaiZe, a ≈2.2B-parameter hybrid (state-space + attention) agentic model built from scratch to serve as a cost-efficient EDA subagent beneath a deployed Flash master.

- **A cost-efficient EDA subagent.** BaiZe receives EDA subtasks in natural language from the master and drives EDA tools in their native scripting languages (PyAether, SKILL, Tcl). Its 56 layers interleave Mamba-2 state-space and MLP layers with four grouped-query-attention layers, so only the latter carry a session-growing state—which is what makes per-subtask cost reduction structural rather than a matter of quantization alone. A staged budget-constrained search (S1–S5) with a reduced-horizon proxy protocol fixes the pre-training recipe. On a 449-task EDA-scripting benchmark spanning three languages and four commercial runtimes, we ask whether it matches deployed Flash masters at comparable accuracy while materially reducing per-subtask cost, and what an on-premise subagent must co-locate with the model.
- **Multimodal grounding for visual EDA artifacts.** Layout, schematic, and waveform views carry much of the correctness signal in custom and analog design, yet a text-only subagent cannot observe them. BaiZe pairs its language core with a from-scratch vision encoder and tests whether a 196-token-per-image budget suffices for such detail-dense views. The encoder reaches 68.2% linear-probe on an internal protocol, yet the MLLM stage averages 26.1% over 11 benchmarks: encoder-quality gains do not transfer downstream at the 224-px single-tile setting.

II. Background and Related Work

A. Small Language Models and Agentic Reasoning

Parameter-efficient SLMs (<2 B) have recently narrowed the gap with larger models on mathematical and commonsense reasoning. MiniCPM [?] introduced scalable training strategies and domain-weighted data mixing for small models; Qwen [?] and Gemma [?] demonstrate that strong code- and math-enriched corpora can lift small models substantially. Most of these efforts, however, treat the final trained model as the deliverable and report comparatively little about the *search* that produced it. Complementarily, agentic instruction-tu-258(g)57[

have been shown to elicit planning and tool-invocation behaviour, yet are rarely paired with a from-scratch pre-training recipe for the 1B regime.

B. Hybrid and State-Space Architectures

State-space models (SSMs) such as Mamba [**?**] and Mamba-2 [**?**] replace full attention with a recurrent scan update, offering linear-time decoding that suits inference-bound deployments. Hybrid designs interleave a small number of attention layers among SSM/MLP layers—notably Nemotron-H [**?**] and Hymba [**?**]—to recover long-range context while retaining fast decode. Released Megatron-Core variants of these families are scarce below 2B, leaving their small-scale trade-offs under-documented; our controlled 2B comparison (Section **??**) addresses precisely this gap.

C. Multimodal Vision-Language Models

Vision-language models connect a vision encoder to a language model through a lightweight projector, enabling end-to-end visual understanding and instruction following. LLaVA [**?**] established visual instruction tuning—aligning a vision encoder to a frozen LLM via a linear projection followed by instruction fine-tuning—as a scalable recipe for open multimodal assistants. LLaVA-OneVision [**?**] consolidates this line into a single open model spanning single-image, multi-image, and video tasks through a staged curriculum: stage-1 projector pre-training, stage-1.5 mid-training, and stage-2 instruction tuning. BaiZe follows this curriculum in stage (iv) (Section **??**), substituting a from-scratch vision encoder (stage iii) for the frozen off-the-shelf encoder.

D. LLMs for Electronic Design Automation

A growing line of work applies LLM agents to chip design tasks such as register-transfer-level generation, verification, and bug triage [**?**], [**?**]. These applications motivate a cheap, always-on, agentic multimodal core that can run per-team or per-engineer; our work supplies the efficient multimodal backbone such systems require.

III. Architecture and Pipeline Overview

BaiZe is an end-to-end multimodal agentic system assembled in five stages: (i) from-scratch pre-training of a hybrid language backbone, (ii) post-training of that backbone (instruction tuning and reinforcement learning to strengthen agentic capability), (iii) pre-training of a custom vision encoder, (iv) alignment of the two into a multimodal large language model, and (v) semiconductor-domain adaptation via reinforcement learning. This section fixes the shared scaffolding—the two backbones, the overall pipeline, and the global training environment—before each stage is detailed in its own section.

A. Language Backbone

The language backbone is a sequential hybrid model [**?**], [**?**]: 56 transformer layers interleave Mamba-2 state-space layers, four grouped-query-attention (GQA) layers, and MLP layers. The layer allocation follows the Nemotron-H sequential hybrid convention—four attention layers among the 56, the remainder Mamba-2 and MLP—as provided by Megatron-Core's Mamba-hybrid recipe, whose 56-layer pattern we reuse verbatim while halving only the hidden and FFN widths to reach the $\approx$2B scale. BaiZe is not a Nemotron-H model: it is an order of magnitude smaller, contains no mixture-of-experts layers, and is pre-trained from scratch on an independent corpus under a fixed GPU-hour budget. Each Mamba-2 layer uses 8 state-space groups; the four attention layers use 16 query heads and 4 query groups (GQA) with untied word embeddings. Crucially, only the four attention layers carry rotary position embeddings (RoPE); the 52 non-attention layers (24 Mamba-2 SSM and 28 MLP) are position-agnostic, so the backbone's long-context extrapolation quality depends on just $4/56{=}7.1\%$ of layers, an asymmetry exploited in Section **??**. The full configuration is listed in Table **??**; the model totals $\approx$2.22B parameters.

TABLE I
BaiZe language-backbone configuration (Mamba2-hybrid).

Hyper-parameter	Value
Transformer layers	56
Architecture	hybrid (Mamba-2 + GQA + MLP)
Mamba-2 / MLP layers	24 / 28
Attention layers	4 (interleaved)
Hidden size	2048
FFN hidden size	8192
Attention heads (Q)	16
Query groups (GQA)	4
Mamba-2 state-space groups	8
Vocabulary size	129,281
Sequence length	$16{,}384$
Activation	SiLU / SwiGLU
Normalisation	RMSNorm
Precision	FP8 tensorwise
Parameters	$\approx$2.22B

B. Vision Encoder

The vision encoder is pre-trained from scratch in stage (iii) and is selected from iso-parameter candidates at the 500–600 M scale; Section **??** reports the comparison and its outcome.

C. Pipeline and Global Environment

Table **??** summarises the five-stage pipeline. Stage (i) production training runs on two $8\times$NVIDIA H100 nodes (16 GPUs; CUDA 12.8, PyTorch 2.8.0, Python 3.10); other stages use a single $8\times$H100 node. Fused kernels, selective activation recomputation, and cross-entropy loss fusion are enabled where applicable, with

TABLE II
BAIZE FIVE-STAGE PIPELINE OVERVIEW.

Stage	Task	Status	Section
(i)	LM pre-training	in progress	IV
(ii)	Post-training (SFT, RLVR)	design	V
(iii)	Vision encoder	complete	VI
(iv)	MLLM alignment	in progress	VII
(v)	Domain adapt. (RLHF)	design	VIII

A. Architecture Selection

We adopt the hybrid backbone because a matched from-scratch comparison at ≈2B scale favours it over a dense transformer (MiniCPM5-2B [14]): lower loss with 11.6% fewer parameters (≈2.22B vs. 2.51B) at ∼8% throughput cost (Table V); the Megatron-Core provider (Section III-A) makes it executable without custom kernels.

B. Training Framework and Learning-Rate Schedule

We use NVIDIA NeMo [15] with Megatron-Core's [16] Mamba-hybrid provider (Section III-A); hyper-parameter searches run at a reduced iteration count with short-horizon ranking transferred to full scale.

BaiZe trains under a WSD schedule (LR 1×10^{-3}, $2{,}000$ warmup $= 0.29\%$ of 700k; search found 250 optimal at 5k but the proportion narrows at full scale) at sequence $16{,}384$ ($613{,}992$ tok/s; $64\times$ extrapolation to 1M). Decay has two segments: A anneals at 16K on instruction data (60–70%), B extends on a 32K→64K→128K ladder (30–40%), with $lr_{\mathsf{min}}{=}1 \times 10^{-4}$.

C. Search Protocol

The recipe was selected via a staged single-variable protocol at 5000 steps: S1 baseline; S2 LR sweep (2×10^{-4}–1×10^{-3}, seven points); S3 schedule axes (WSD vs. cosine, decay ratio, warmup, min_lr); S4 annealing mix; S5 long run + three seeds. Total budget ≈82 GPU-hours on eight H100s; losses are proxy measurements for ranking.

D. Data

The stable phase uses a seven-source mixture yielding a $74{:}19{:}7$ web/code/math split (0.74 `Ultra-FineWeb`, 0.19 `UltraData-Code`, 0.07 `UltraData-Math`); a 400-trial Bayesian search confirmed this region optimal within 0.6 pp. Sources are pre-blended into large shards ($\mathcal{O}(10)$ vs. $\mathcal{O}(600)$ index files per rank), bounding the data-loader footprint. Contamination is controlled by a $193{,}295$-entry 13-gram blacklist from 536 benchmarks; shards are scanned for ≥ 0.80 n-gram overlap.

E. Long-Context Data Gate

Only 2.52% of tokens sit in documents $\geq$32K and 0.03% at $\geq$128K (mean 889 tokens; Table III), so we measured the distribution before budgeting segment B. We gate the ladder: $\geq$32K share must reach 20% and $\geq$128K 5% (else it stops at 64K), with EOD resets and de-duplication.

TABLE III
LONG-CONTEXT DATA GATE (LEFT) AND TRAINING LADDER (RIGHT). PER-GPU TOKENS HELD AT $16{,}384$; 32K THROUGHPUT MEASURED, 64K/128K EXTRAPOLATED.

Thresh.	Docs	Share	Status	Rung	Seq.	CP/DP	Tput
$\geq$32K	4,257	**2.52%**	not met	1	32K	2/8	$351{;}526$
$\geq$128K	—	**0.03%**	not met	2	64K	4/4	≈180K
$\geq$1M	—	n/a	n/a	3	128K	8/2	≈90K

TABLE IV
PRODUCTION BUDGET. WALL-CLOCK IS ESTIMATED AT MEASURED THROUGHPUT.

Phase	Tokens	Steps	Wall-clock	Status
Warmup + stable	2.94T	700k	≈56 d	30.7k done
Decay A (16K, 60–70%)	58.7B	14k	≈1.2 d	queued
Decay B (LC, 30–40%)	25.2B	6k	≈2.3 d	gated
Total	**3.02T**	**720k**	≈59.5 d	—

F. Results and Production Run

The search selects WSD at LR 1×10^{-3} with $86{:}10{:}4$ annealing, reaching validation loss `2.2054` at 20000 steps (2.6739 ± 0.0469 across three seeds at 5000; Table V).

The recipe is scaled to 2.94T tokens at `DP`$=16$ over two $8\times$H100 nodes, `GBS`$=256$ at sequence $16{,}384$ ($4{,}194{,}304$ tokens/step), FP8 tensorwise, AdamW. At the time of writing, the run has passed iteration $30{,}660$ of $700{,}000$ at ≈598K tok/s, training loss 2.151, no NaN (Table IV). The $613{,}992$ tok/s peak is FP8 at $16{,}384$; shorter sequences use BF16.

G. Long-Context Training in the Decay Tail

Long-context training is segment B of the decay: the LR is already low, and neither SFT nor RL builds retrieval ability at this scale. The decay budget is $\sim 2.9\%$ of main volume (83.9B / $20{,}000$ steps); long-context data is 30–40% (25–34B). The ladder runs 32K→64K→128K at `CP` $\in \{2, 4, 8\}$, $16{,}384$ tokens/GPU, continuing decay without re-warming; $\theta{=}10^6$ fixed, so 128K is trained and 1M is $8\times$ extrapolation (Table III). Reference cost ~ 880 GPU-hours (Table IV). At WSD end: one snapshot—needle retrieval (passkey + RULER, target $\geq 95\%$ at 1M) and perplexity extrapolation ($\leq 1.10\times$).

H. Long-Context Capability and Inference Cost

With $\theta{=}10^6$ on the four attention layers, perplexity evolves $83.2 \to 51.5 \to 53.3 \to 53.0 \to 55.8 \to 55.4$ from 4K to 1M, flat beyond 64K; only $4/56{=}7.1\%$ of layers carry RoPE. Under SGLang, the hybrid achieves $39{,}377$ tok/s prefill at 128K and serves 1M in 172s at 34.7 GB; the dense transformer cannot serve beyond 512K on a single 80 GB H100 (KV-pool cap ≈455K). The advantage is architectural: $O(1)$ per-layer state for 52 of 56 layers.

V. LLM POST-TRAINING

Stage (ii) adapts the pre-trained backbone to EDA scripting, and it is the stage that carries BaiZe's domain claim: the pre-training corpus of Section IV-D is general web, code,

TABLE V
ARCHITECTURE COMPARISON (TOP;

TABLE VII
MLLM DOWNSTREAM RESULTS (STAGE-2 STEP 3500, %; INTERNAL

REFERENCES

[1] Y. Liu, S. Hou, X. Chen, Y. Wang, S. Yuan, Q. Gan, S. You, F. Chen, W. Li, S. Hu, Y. Liu, E. Xia, X. Lu, H. Wang, F. Xu, and Y. Li, "Zhulong: Execution-grounded llm agent for eda scripting with offline api self-exploration," 2026.

[2] S. Hu, Y. Tu, X. Han, C. He, G. Cui, X. Long, Z. Zheng, Y. Fang, Y. Huang, W. Zhao *et al.*, "Minicpm: Unveiling the potential of small language models with scalable training strategies," *arXiv preprint arXiv:2404.06395*, 2024.

[3] A. Yang, B. Li, B. Zhang *et al.*, "Qwen3 technical report," 2025.

[4] G. Team, "Gemma 3 technical report," 2025.

[5] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang *et al.*, "Agentbench: Evaluating llms as agents," in *International Conference on Learning Representations (ICLR)*, 2024.

[6] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.

[7] T. Dao and A. Gu, "Transformers are ssms: Generalized models and efficient algorithms through structured state space duality," *arXiv preprint arXiv:2405.21060*, 2024.

[8] NVIDIA, "Nemotron-h: A family of accurate and efficient hybrid mamba-transformer models," 2025.

[9] X. Dong, Y. Fu, S. Diao *et al.*, "Hymba: A hybrid-head architecture for small language models," 2024.

[10] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[11] B. Li, Y. Zhang, D. Guo, R. Zhang, F. Li, H. Zhang, K. Zhang, P. Zhang, Y. Li, Z. Liu, and C. Li, "Llava-onevision: Easy visual task transfer," 2024.

[12] M. Liu, T. E. Ene, R. Kirby, C. Cheng, N. Pinckney, R. Liang, J. Alben, H. Anand, S. Banerjee, I. Bayraktaroglu *et al.*, "Chipnemo: Domain-adapted llms for chip design," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[13] J. Blocklove, R. Chapman, B. Tan, D. Chen *et al.*, "Chipbench: The llm chip design benchmark," 2024.

[14] OpenBMB, "Minicpm5: Openbmb small language model series," https://github.com/OpenBMB/MiniCPM, 2026, model release (1B: 2026-05, 2B: 2026-09); no independent training paper available at time of writing.

[15] NVIDIA, "Nemo: A toolkit for building ai applications using neural modules," https://github.com/NVIDIA/NeMo, 2024, version 2.x; used for pre-training via the NeMo run entry point.

[16] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," 2020.